Supervised Fine-Tuning and GRPO on Qwen3-0.6B for Mathematical Reasoning

GSM8K · SVAMP · MMLU Evaluation

Yue Wu, Hanfu Zhao

March 17, 2026

Outline

1. Motivation & Objective
2. Literature Review
3. Fine-tuning Method
4. Pipeline
5. Datasets
6. Results
7. Additional Experiments
8. Summary

Motivation & Objective

Question: Can a tiny 0.6B model become a reliable math problem solver through fine-tuning alone?

Why it matters

- Small models are cheap to run and deploy
- Math reasoning is a hard, structured task — a good benchmark for fine-tuning quality
- GRPO is a lightweight RL alternative to RLHF — no value network needed

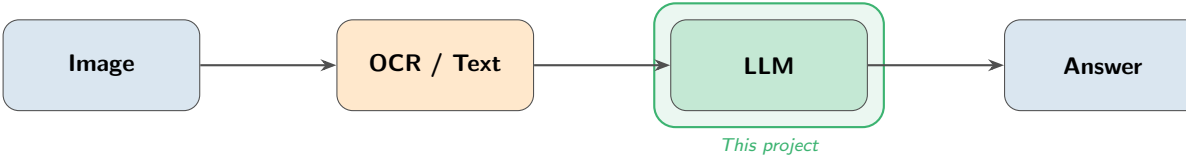
This project

- Train **Qwen3-0.6B-Base** with SFT then GRPO
- Evaluate on GSM8K, SVAMP, and MMLU
- Use LoRA / PEFT to keep VRAM under 15 GB

At a Glance

Model	Qwen3-0.6B-Base
SFT method	DoRA (r=8, α=16)
LoRA ablation	LoRA (r=8, α=16)
SFT steps	300
GRPO steps	200
Train data	GSM8K (7 473 ex.)
Eval data	GSM8K / SVAMP / MMLU
Hardware	2× NVIDIA T4 (15 GB)
Framework	HuggingFace + TRL

Application Story: Photo-to-Answer Pipeline



Real-world scenario — Yuansouti ()

- Students photograph a math problem → OCR extracts the text → an LLM solves it step by step → answer is returned.
- This project focuses on the LLM stage: fine-tuning **Qwen3-0.6B** so it can reliably solve grade-school arithmetic from the extracted text.

Literature Review: Math Reasoning in LLMs

Recent work suggests that strong mathematical reasoning in large language models (LLMs) arises from combining **math-specific supervision** with **reinforcement learning (RL)**. However, different studies assign different roles to each stage.

- **DeepSeekMath (Shao et al., 2024)**
  - Builds a 7B open math model using math-focused continued pretraining, instruction tuning, and GRPO-based RL.
  - Shows that relatively small models can achieve strong reasoning performance with specialized training.
  - Demonstrates strong results on benchmarks such as **MATH (51.7%)** and **GSM8K**.
- **Two-Stage Recipe (Yoshihara et al., 2025)**
  - Proposes a sequential training strategy: **extended SFT** followed by **GRPO**.
  - Studies the distinct contributions of supervised learning and reinforcement learning.

Both works highlight the importance of combining supervised learning with RL, but differ in how much each stage contributes to reasoning ability.

Literature Review: Role of SFT vs RL

The two papers offer different interpretations of how reasoning ability emerges.

- **DeepSeekMath Perspective**
  - RL is a key ingredient for improving reasoning in open models.
  - GRPO enables efficient training with verifiable rewards from math problems.
  - However, improvements come from multiple factors:
    - math-domain pretraining corpus
    - instruction tuning
    - RL optimization

    making attribution difficult.
- **Two-Stage Recipe Perspective**
  - **Extended SFT** (up to 10 epochs) is the main driver of accuracy.
  - **GRPO** mainly reduces reasoning length and improves token efficiency.
  - RL acts more as a **behavior optimizer** than a capability creator.

Takeaway: Domain data and SFT build reasoning capability, while RL helps refine reasoning trajectories and efficiency.

Fine-tuning Methods (1/2) — SFT

SFT Supervised Fine-Tuning

- Cross-entropy training on (prompt, chain-of-thought) pairs
- Prompt tokens masked — only the *response* contributes to the loss
- Teaches the model *what format* to use and *how to reason step-by-step*
- Fast and stable; used as the starting point for GRPO

Fine-tuning Methods (2/2) — GRPO

How GRPO works — step by step

1. Sample $G = 8$ **completions** per prompt
2. Score each: correct $+1.0$, #### $+0.5$
3. Advantage = score relative to group mean: $\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r}$ (above mean $\rightarrow +$, below $\rightarrow -$)
4. Reinforce high-scorers, suppress low-scorers
5. KL penalty keeps model close to DoRA-SFT reference

Why not PPO?

- PPO needs a separate **value network** — doubles memory
- GRPO uses the **group of 8 as its own baseline**
- Cheaper; ideal for verifiable tasks (math, code)

Example (G=8 for one prompt)

Completion	Reward
`#### 13` (correct)	1.5
`#### 16` (wrong)	0.5
`answer is 13`	1.0
`I don't know`	0.0

Mean $\approx 0.75 \Rightarrow$ row 1 & 3 boosted

Pipeline (1/3) — Training

GSM8K
7 473 ex.

Qwen3-0.6B Base pre-trained, no task knowledge

300 steps · fp16 · effective batch 32 · DoRA r=8

DoRA-SFT Model — DoRA adapter learns step-by-step reasoning

200 RL steps · G=8 completions · KL penalty $\beta = 0.01$

GRPO Model — merged full model RL fine-tuning on top of DoRA-SFT

Pipeline (2/4) — Evaluation: Model Loading

All **3 models** are evaluated on all **3 datasets** — 9 runs in total. The evaluation script and prompts are identical; only the loaded weights differ.

How each model is loaded

- **Base** — load `Qwen3-0.6B-Base` directly
- **DoRA-SFT** — load base, apply the DoRA adapter, merge weights
- **LoRA-SFT** — same as above but with a plain LoRA adapter (ablation only)
- **GRPO** — load the merged full model directly (base + DoRA-SFT + GRPO all baked in; no adapter needed)

Metrics recorded per example

- **Correct** — predicted answer matches gold label
- **Format** — output contains `#### N`
- **Eval score** — correctness (+1.0) + format bonus (+0.5)

Pipeline (3/4) — Evaluation: Answer Grading

GSM8K & SVAMP (open-ended)

- Generate up to 256 tokens (greedy)
- Extract number: look for `#### N` first, then "Final answer: N", then last number in text
- Normalize (strip commas, drop `.0`) and compare

MMLU (multiple choice)

- Generate only 16 tokens
- Extract first A/B/C/D letter from output
- Compare to the correct option letter

Pipeline (4/4) — Analysis & Outputs

After all evaluations are complete, `analyze.py` reads the results from all nine JSONL files and the training logs, then automatically generates plots and a summary table.

Training diagnostics

- SFT cross-entropy loss curve over 300 steps
- GRPO mean reward, KL divergence, and policy-gradient loss over 200 RL steps

Evaluation results

- Accuracy and format rate for all three models on GSM8K
- Side-by-side GSM8K vs SVAMP accuracy comparison
- MMLU accuracy per subject to check for forgetting
- Answer length and composite score distributions

All outputs are saved to `images/` and zipped for download. No manual post-processing is needed — the full analysis runs in under two minutes after training completes.

Datasets (1/2) — Overview

GSM8K

- 8.5K grade-school word problems
- Train: 7 473 · Test: 1 319 (eval: 500)
- Chain-of-thought ending with `#### N`
- **Purpose**: in-distribution training & eval

SVAMP

- 1 000 adversarial math problems
- Same difficulty as GSM8K but different phrasing
- Eval: 500 examples
- **Purpose**: out-of-distribution generalisation

MMLU (subset)

- 2 subjects, 150 each (300 total):
  - High School Mathematics
  - Elementary Mathematics
- **Purpose**: catastrophic forgetting check

Datasets (2/2) — Examples

GSM8K

```
Q: Janet has 16 eggs.  She eats
3 for breakfast.  How many left?

A: Let's think step by step.
16 - 3 = 13 eggs left.
#### 13
```

Subtraction problem. Answer marked with ####. Used for training and in-distribution eval.

SVAMP

```
Q: Janet has 16 eggs.  She buys
3 more.  How many does she have?

A: Let's think step by step.
16 + 3 = 19 eggs.
#### 19
```

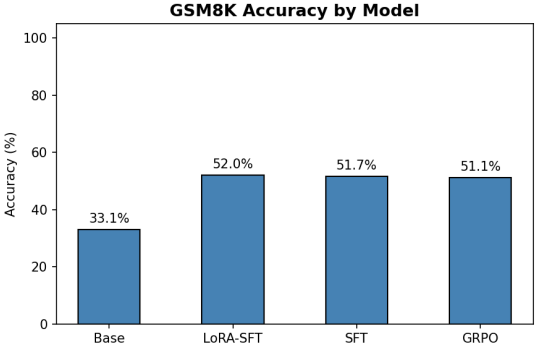
Same numbers, different operation & phrasing. Tests true understanding, not pattern-matching.

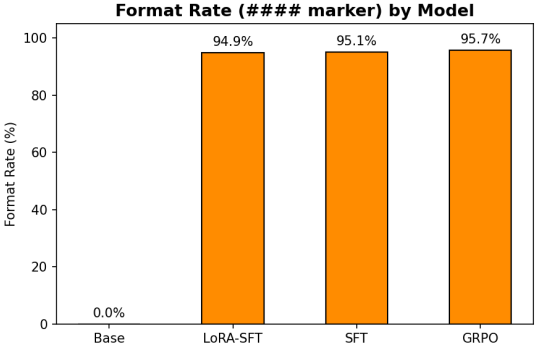
MMLU

```
Q: What is the value of
log2(8)?

(A) 2   (B) 3
(C) 4   (D) 6

Answer:  B
```

Multiple-choice, no chain-of-thought. Only the letter (A/B/C/D) is extracted. Used to check for catastrophic forgetting.

GSM8K Results — In-Distribution Math (1/2)





	Base	LoRA-SFT	DoRA-SFT	GRPO
Accuracy	33.1%	**52.0%**	51.7%	51.1%
Format rate	0.0%	94.9%	95.1%	**95.7%**
Eval score	0.33	0.99	0.99	0.99

GSM8K Results — In-Distribution Math (2/2)

Base — 33.1%

- No GSM8K training
- Freeform output — no ####
- Answer extractor misses most correct calculations

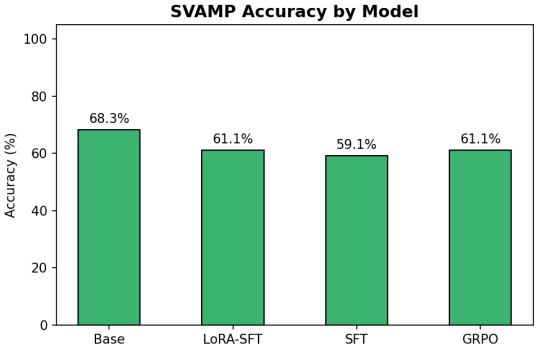
SFT — 52.0% (+18.9 pp)

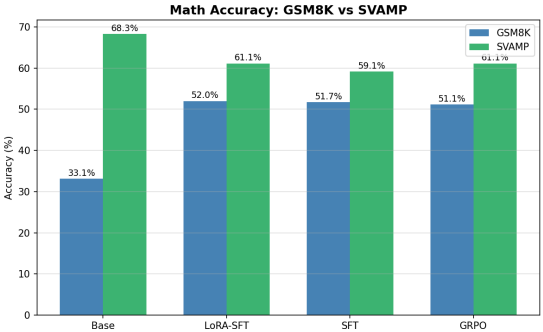
- 7 473 (question, CoT) pairs
- Directly learns output format & reasoning steps
- Most targeted signal → biggest jump

GRPO — 51.1% (−0.6 pp vs DoRA-SFT)

- Trained on top of the DoRA-SFT checkpoint
- Only 200 noisy RL steps — marginal accuracy drop
- Highest format rate (95.7%) — RL optimises the reward signal

SVAMP Results — Out-of-Distribution Generalisation (1/2)





	Base	LoRA-SFT	DoRA-SFT	GRPO
SVAMP Acc.	**68.3%**	61.1%	59.1%	61.1%

SVAMP Results — Out-of-Distribution Generalisation (2/2)

Base — 68.3% (highest)

- Never trained on any fixed format
- Handles varied phrasing flexibly
- Nothing to "unlearn" $\rightarrow$ best OOD score

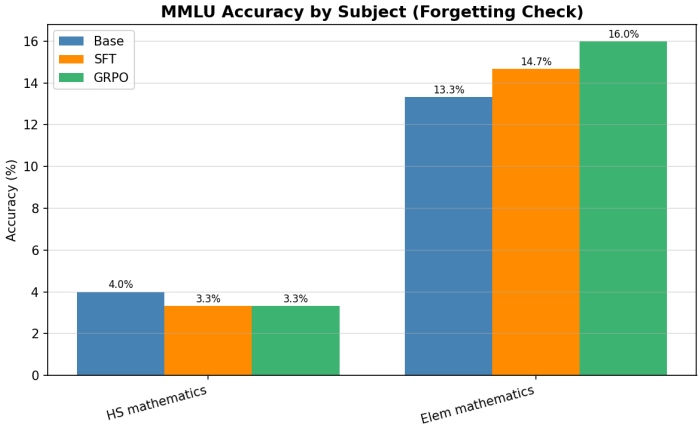
SFT — 59–61% (-7 to -9 pp)

- GSM8K training biases toward one question style
- SVAMP phrasing breaks that expectation
- Classic *in-distribution overfit*

GRPO — 61.1% ($+2$ pp)

- Reward checks the *output answer*, not the *input phrasing*
- Model is not penalised for varied question wording
- Partially recovers the OOD drop

MMLU Results — Catastrophic Forgetting Check (1/2)



Model	Overall	Δ
Base	8.7%	—
SFT	9.0%	+0.3 pp
GRPO	9.7%	+1.0 pp

- All models score ∼8.7–9.7%
- **Below random chance** (25%)
- SFT score unchanged from Base — **no forgetting**

MMLU Results — Catastrophic Forgetting Check (2/2)

Why ∼9% across the board

- MMLU requires broad academic knowledge
- 0.6B parameters lacks the capacity
- **Model-size ceiling**, not a training issue

Why SFT does not hurt

- LoRA only updates low-rank adapters
- Original pre-trained weights stay frozen
- Arithmetic skill added, nothing erased

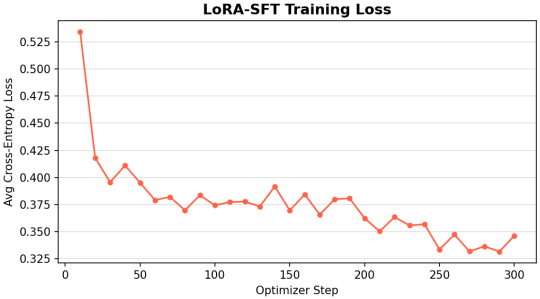
Why GRPO gains +1.0 pp

- Reward requires correct *and* formatted output
- Model reuses SFT's structured reasoning style to earn reward
- Transfers to Elementary Math (13.3%→16.0%) and overall (8.7%→9.7%)

SFT Training Curves

DoRA-SFT

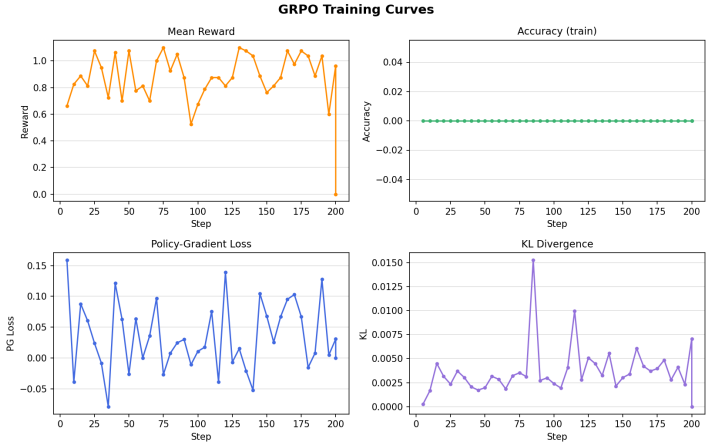
LoRA-SFT



SFT Details

300 optimiser steps · Batch size 2, grad accum 16 (effective batch = 32) · fp16 mixed precision · Gradient checkpointing · Loss decreases steadily for both variants.

GRPO Training Curves



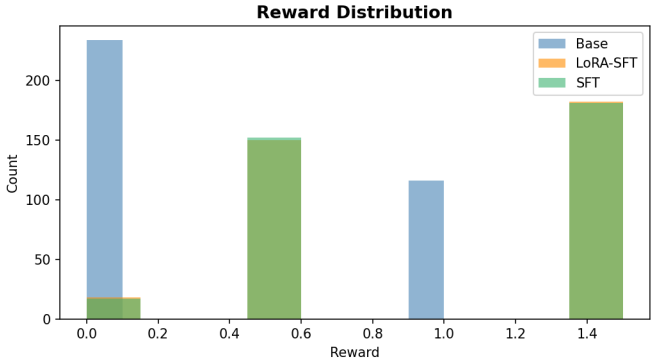
RL Training Reward

Condition	Reward
Correct + ####	1.5
Correct only	1.0
#### only	0.5
Neither	0.0

Used *only* during GRPO training to update the policy.

- Mean training reward rises **0.66 → 0.96**
- High variance is normal — group sampling is noisy
- KL divergence stays small — model stays close to DoRA-SFT reference

Composite Eval Score Distribution



Composite eval score: +1.0 if correct +0.5 if #### (max 1.5)

Same formula as the GRPO training reward, but applied post-hoc at inference to *all* models — not a training signal for Base or SFT.

Base — peak at 0

- No #### convention learned
- Most outputs wrong *and* unformatted

DoRA-SFT — peak near 1.0

- Learns correct answers *and* format together
- Distribution shifts sharply rightward

GRPO — similar to DoRA-SFT

- Starts from DoRA-SFT checkpoint
- RL training used same formula as reward signal
- Pushes slightly more answers to max score (1.5)

Additional Experiments — Overview

Three further experiments to address key design decisions required by the project brief.

1. **Prompt Design Comparison**
   Does the prompt format matter? Compare three strategies.
2. **DoRA vs LoRA**
   Does Weight-Decomposed LoRA outperform standard LoRA?
3. **Step-by-Step Reasoning Validity**
   Are the intermediate arithmetic steps actually correct?

All experiments use the same Qwen3-0.6B model, GSM8K dataset, and hyperparameters for fair comparison.

Experiment 1 — Prompt Design Comparison (1/2)

Three Strategies

- **direct** Bare question, no guidance
- **cot** "Let's think step by step" (Chain-of-Thought)
- **rules** Explicit math rules ($+$, $-$, $\times$, $\div$) then step-by-step

Why it matters

- Prompt design is a key practical decision
- Same model weights — only the prompt changes
- Tests whether explicit instructions help a small model

Prompt Examples

direct:

```
Question: Janet has 16 eggs...
Answer:
```

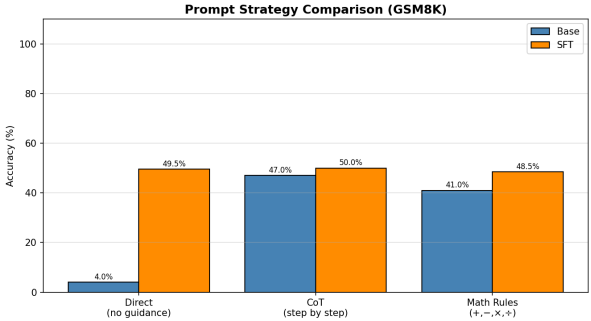
cot:

```
You are a helpful assistant...
Question: Janet has 16 eggs...
Answer: Let's think step by step.
```

rules:

```
You are a math tutor. Rules:
- Addition (+): combining...
- Subtraction (-): difference...
Question: Janet has 16 eggs...
Answer: Let's solve step by step.
```

Experiment 1 — Prompt Design Comparison (2/2)



Strategy	Base	SFT
direct	4.0%	49.5%
cot	**47.0%**	**50.0%**
rules	41.0%	48.5%

Key findings

Direct (base) — 4%

- No guidance → no reasoning signal
- Model outputs random text, rarely correct

CoT — best for base (+43 pp)

- Activates pre-trained chain-of-thought knowledge
- Biggest boost for an untuned model

Rules — diminishing returns

- Explicit rules add noise, not clarity
- After SFT, all prompts converge (∼49–50%)
- SFT already internalised the reasoning format

Experiment 2 — DoRA vs LoRA (1/2)

What is DoRA?

- LoRA: $W' = W + BA \quad (B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k})$
- DoRA decomposes weight into **magnitude** and **direction**: $W' = m \cdot \frac{W+BA}{\|W+BA\|}$
- Extra learnable magnitude scalar m per layer
- Inspired by Weight Normalization (Salimans & Kingma, 2016)

Experimental Setup

	LoRA / DoRA
Rank r	8
Alpha α	16
Target modules	all attn + MLP
Steps	300
Batch size	$2 \times 16 = 32$
Precision	fp16

Identical hyperparameters — only `use_dora` *flag differs.*

Experiment 2 — DoRA vs LoRA (2/2)

Results

Method	GSM8K	Format	SVAMP
LoRA (SFT)	**52.0%**	94.9%	**61.1%**
DoRA (SFT)	51.7%	**95.1%**	59.1%

Why the gap is so small

DoRA's extra magnitude scalar m is most beneficial when the adaptation requires both scaling *and* rotating the weight matrix. For a simple task like grade-school arithmetic at rank $r = 8$, the direction update alone in standard LoRA is already sufficient.

Interpretation

Why LoRA matches DoRA:

- GSM8K requires learning a narrow skill (arithmetic + output format) — low intrinsic rank
- At 0.6B scale the weight matrices are small; the magnitude dimension adds little expressive power
- 300 steps may not be enough to utilise DoRA's extra degree of freedom

Takeaway

For sub-1B models on narrow tasks, standard LoRA is sufficient. DoRA's benefit is more likely to appear at larger scale or higher rank.

Experiment 3 — Step-by-Step Reasoning Validity (1/2)

Motivation

Final-answer accuracy only tells us *what* the model got right, not *how*. A model could reach the right answer through wrong intermediate steps (lucky cancellation).

Method

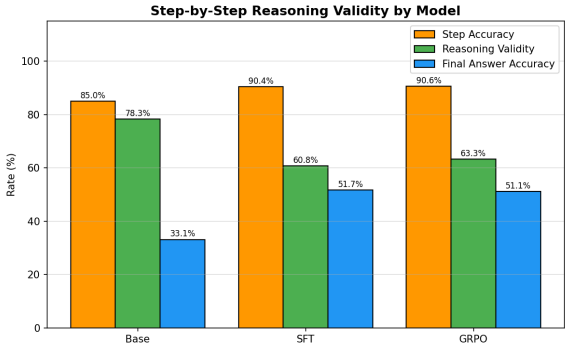
- Extract every arithmetic expression from the completion
  e.g. `16 - 3 = 13`, `5 * 4 = 20`
- Verify each calculation independently with Python
- Classify each completion into a 2×2 quadrant:

	Correct answer	Wrong answer
Valid reasoning	**Best**	Unlucky
Invalid reasoning	Lucky	**Worst**

Metrics

- **Step accuracy**: fraction of arithmetic steps that are mathematically correct
- **Reasoning validity rate**: fraction of completions where *all* steps are correct
- **Avg steps**: reasoning depth per completion

Experiment 3 — Step-by-Step Reasoning Validity (2/2)



	Base	SFT	GRPO
Step accuracy	85.0%	90.4%	**90.6%**
Validity rate	**78.3%**	60.8%	63.3%

Base — highest validity (78.3%)

- Short, simple responses
- Fewer steps → fewer chances to err
- High validity despite low final accuracy

SFT — lower validity (60.8%)

- Longer, detailed reasoning chains
- More steps → more arithmetic opportunities for error

GRPO — slight recovery (63.3%)

- Reward penalises wrong answers
- Indirectly encourages more careful arithmetic
- Step accuracy up +0.2 pp vs SFT

Summary & Discussion

What We Achieved

- Fine-tuned Qwen3-0.6B on 2×T4 GPUs
- **SFT**: +18.9 pp on GSM8K, 95% format adoption
- **GRPO**: reward rises 0.66→0.96, format improves further
- **No catastrophic forgetting** on MMLU
- **Prompt comparison**: CoT dominates for base (+43 pp)
- **DoRA vs LoRA**: nearly identical (< 0.5 pp)
- **Reasoning validity**: SFT/GRPO improve step accuracy (+5.5 pp)

Limitations & Future Work

- GRPO slightly below DoRA-SFT (−0.6 pp): more RL steps needed
- SVAMP drop (−9 pp): format over-specialisation on GSM8K style
- MMLU: 0.6B too small; 3B+ needed for meaningful scores
- **Next steps**:
  - More GRPO steps (500+)
  - Mix SVAMP into training data
  - Try Qwen3-1.7B or 3B

Summary

	GSM8K		SVAMP	MMLU
Model	Acc.	Format	Acc.	Acc.
Base	33.1%	0.0%	**68.3%**	8.7%
LoRA-SFT	**52.0%**	94.9%	61.1%	—
DoRA-SFT	51.7%	95.1%	59.1%	9.0%
GRPO	51.1%	**95.7%**	61.1%	**9.7%**

LoRA-SFT and DoRA-SFT differ by < 0.5 *pp on GSM8K. GRPO closes the SVAMP gap vs DoRA-SFT.*

Thank You

Questions?

Model	Qwen3-0.6B-Base (Alibaba Cloud)
Dataset	GSM8K, SVAMP, MMLU
Code	github.com/202520030411/Fine-tuning-and-GRPO-on-Qwen
Tools	HuggingFace Transformers, PEFT, TRL, PyTorch